



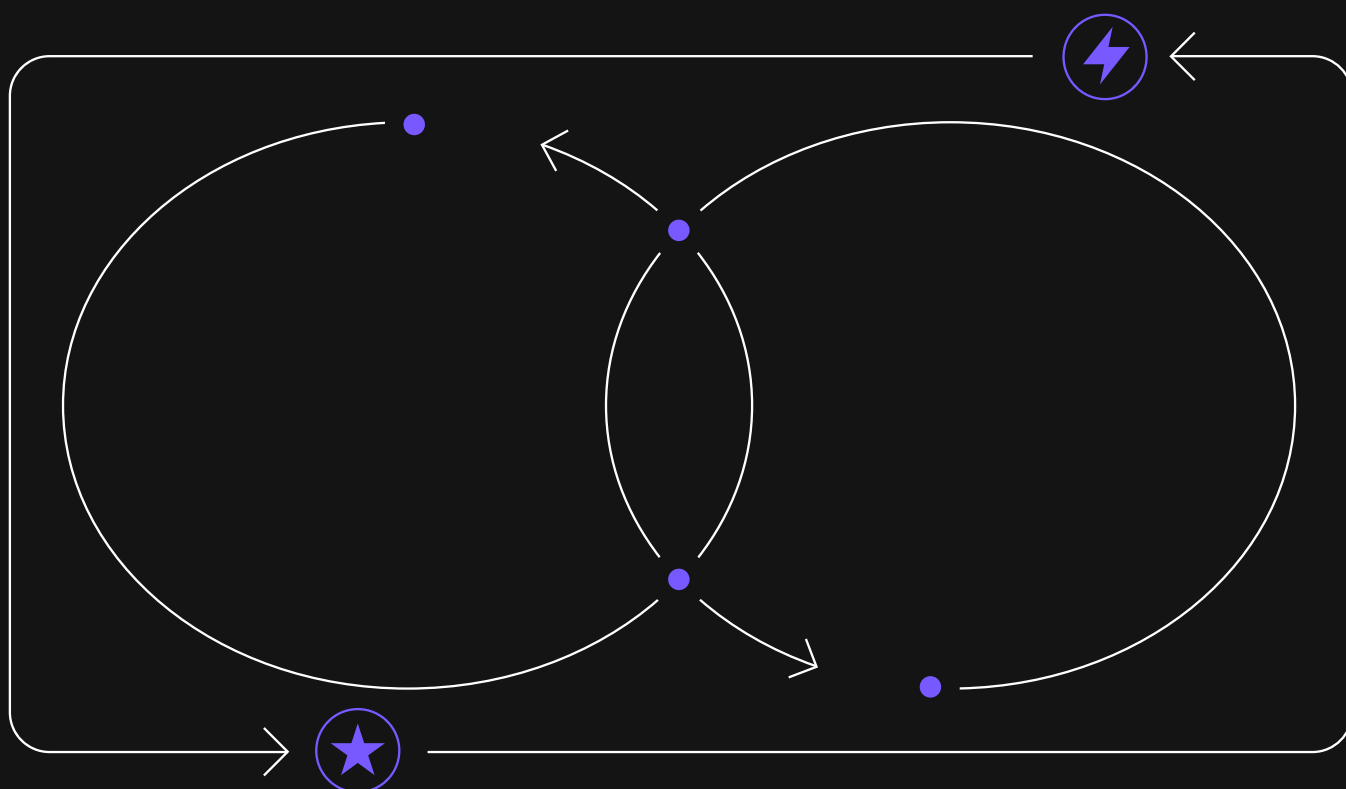
Машинное обучение

Лонгрид

Методы оптимизации

02

Матричное дифференцирование и одномерная оптимизация



Матрично-векторное дифференцирование

Контекст

Все методы оптимизации, кроме самых примитивных, работают не с самой функцией, а с её локальной линейной или квадратичной моделью: градиентный спуск идёт вдоль антиградиента, метод Ньютона минимизирует квадратичную аппроксимацию. Значит, прежде чем обсуждать методы, надо уметь эти модели выписывать, то есть дифференцировать функции от векторов и матриц.

Применение

Рабочий инструмент — язык дифференциалов: он не требует расставлять индексы, одинаково работает для вектора и для матрицы и превращает вычисление градиента в цепочку из пяти-шести стандартных правил. Именно так считаются градиенты функций потерь в машинном обучении, и именно эту механику воспроизводит автоматическое дифференцирование внутри `torch` и `jax` — понимая её, ты знаешь, что именно и с какой сложностью считает фреймворк.

Задача лонгрида

Отработаем технику: сначала на функциях векторного аргумента, где ответ можно проверить по координатам, затем на матричных.

Краткий план

- Градиент, гессиан, якобиан: что проверять
- Дифференциалы в деле

1. ГРАДИЕНТ, ГЕССИАН, ЯКОБИАН



Размерности стоит проверять всегда и первым делом: $\nabla f(x)$ живёт в том же пространстве, что и x ; якобиан отображения $\mathbb{R}^n \rightarrow \mathbb{R}^m$ имеет размер $m \times n$; гессиан скалярной функции — это якобиан её градиента, то есть $n \times n$. Большая часть ошибок в матричных выкладках ловится одной проверкой согласованности размеров, и делать её надо до того, как выкладка закончена.

ПРИМЕР 1

Аппроксимации Тейлора — это в точности то место, где рождаются методы. Минимизируя квадратичную модель по h при $\nabla^2 f(x) \succ 0$, получаем

$$h = -(\nabla^2 f(x))^{-1} \nabla f(x)$$

— шаг метода Ньютона. Минимизируя линейную модель при ограничении $\|h\|_2 \leq \delta$, получаем шаг вдоль антиградиента. Дальше весь курс — это варианты выбора модели и способа её минимизации, поэтому уметь выписывать ∇f и $\nabla^2 f$ нужно механически.

2. ДИФФЕРЕНЦИАЛЫ

Дальше техника одна и та же: выписываем приращение, оставляем главную линейную по H часть, приводим её к виду «скалярное произведение чего-то на H » и это «что-то» объявляем градиентом. Разница только в том, в каком пространстве живёт H : в $\mathbb{R}^n$ или в $\mathbb{R}^{m \times n}$ со скалярным произведением $\langle A, B \rangle = \text{tr}(A^\top B)$.

ЗАМЕЧАНИЕ

Отдельный сюжет — стоимость. Гессиан функции n переменных — это n^2 чисел, и при $n \sim 10^6$ его нельзя ни хранить, ни считать. Зато произведение $\nabla^2 f(x) v$ часто вычисляется за ту же цену, что и один градиент: это ровно та оракульная логика, которую мы обсуждали для матриц. Поэтому, найдя гессиан, полезно сразу спросить себя, какова структура ответа — диагональ плюс поправка малого ранга встречается чаще, чем плотная матрица.

ЗАДАЧА 1

Найди градиент функции $f(X) = \log \det X$, определённой на множестве симметричных положительно определённых матриц $\mathbb{S}_{++}^n$.

ЗАДАЧА 2

Рассмотрим функцию **log-sum-exp**

$$f(x) = \ln \left(\sum_{i=1}^n e^{x_i} \right).$$

Найди $\nabla f(x)$ и $\nabla^2 f(x)$.

ЗАДАЧА 3

Функция $f(x) = \ln \langle x, Ax \rangle$ определена там, где $\langle x, Ax \rangle > 0$. Найди $\nabla f(x)$ и $\nabla^2 f(x)$ для произвольной $A \in \mathbb{S}^n$.

3. ЕСЛИ КРАТКО

- Проверка размерностей — самый дешёвый способ поймать ошибку: градиент лежит там же, где аргумент, якобиан отображения $\mathbb{R}^n \rightarrow \mathbb{R}^m$ имеет размер $m \times n$, гессиан — $n \times n$ и симметричен.
- Универсальная техника: выписать дифференциал, привести его к виду $\langle \cdot, H \rangle$, прочитать градиент. Для матричного аргумента скалярное произведение — это $\text{tr}(A^\top B)$.
- Правила $d(\text{tr } X)$, $d(\det X)$, $d(X^{-1})$ вместе с правилом цепочки закрывают практически все матричные примеры, включая $\log \det X$.
- Найдя гессиан, стоит посмотреть на его структуру: если это диагональ плюс малоранговая поправка, работать с ним можно и при огромных n .

Одномерная оптимизация: точные методы

Контекст

Практически любой метод оптимизации устроен как «выбрать направление, затем выбрать длину шага вдоль него». Второй подзадачей и является одномерная минимизация: для фиксированных x и d мы минимизируем функцию одной переменной $\varphi(\alpha) = f(x + \alpha d)$. Она проста настолько, что позволяет разобрать на пальцах всё, что дальше будет происходить в многомерном случае: унимодальность вместо выпуклости, скорость сокращения отрезка вместо скорости сходимости, компромисс между надёжностью и скоростью.

Применение

Одномерные методы — это ещё и полноценный инструмент сам по себе: подбор параметра регуляризации, калибровка порога, поиск корня производной. Важно, что все

они опираются только на возможность вычислить $\varphi(\alpha)$ в точке — никакие производные не нужны, а значит, метод применим и к результату сложной симуляции, у которой производных попросту нет.

Задача лонгрида

Научимся считать стоимость метода в вызовах функции, а не в итерациях, и разберём метод Брента — то, что реально стоит за вызовом одномерного оптимизатора в библиотеке.

Краткий план

- Считаем вызовы функции, а не итерации
- Метод Брента

1. СЧИТАЕМ ВЫЗОВЫ ФУНКЦИИ, А НЕ ИТЕРАЦИИ



Сравнивать методы сокращения отрезка надо не по числу итераций, а по числу вычислений функции — именно они дороги, а всё остальное в этих методах бесплатно. Дихотомия сокращает отрезок в 2 раза за 2 вызова, то есть в $\sqrt{2} \approx 1,41$ раза за вызов; золотое сечение — в $1/\tau \approx 1,618$ раза за вызов, потому что одна из двух пробных точек переиспользуется на следующем шаге. При равном бюджете вызовов золотое сечение выигрывает, хотя «за итерацию» оно сокращает отрезок слабее.

Оба метода сходятся линейно: длина отрезка неопределённости убывает как геометрическая прогрессия, и число вызовов до точности ε растёт как $O(\log(1/\varepsilon))$. Это надёжно, но медленно, и цена надёжности здесь ровно одна — методы не используют никакой информации о гладкости функции, кроме сравнения значений.

ЗАДАЧА 1

Пусть на отрезке $[a, b]$ длины L_0 ищется минимум унимодальной функции.

1. Вырази длину отрезка неопределённости после N вычислений функции для метода дихотомии (считай $\delta \rightarrow 0$) и для метода золотого сечения.
2. Сколько вычислений функции нужно каждому методу, чтобы сократить отрезок в 10^6 раз?
3. Почему золотое сечение выигрывает, хотя за одну итерацию сокращает отрезок слабее? Какое свойство числа $\tau = \frac{\sqrt{5}-1}{2}$ за это отвечает?

2. МЕТОД БРЕНТА

Параболическая интерполяция вблизи невырожденного минимума сходится сверхлинейно — гораздо быстрее золотого сечения. Но гарантий у неё нет никаких: при неудачном расположении точек знаменатель в формуле вершины обращается в ноль, вершина оказывается максимумом или вылетает за пределы отрезка.

ОПРЕДЕЛЕНИЕ 1

Метод Брента — комбинация этих двух идей. На каждом шаге пробуются параболическая интерполяция по трём лучшим известным точкам; полученная вершина u принимается, только если выполнены условия безопасности:

- точка u попадает внутрь текущего отрезка неопределённости;
- сделанное смещение меньше половины смещения на шаге, сделанном перед предыдущим.

Если хотя бы одно условие нарушено, делается обычный шаг золотого сечения.

ЗАМЕЧАНИЕ

Второе условие выглядит странно, но именно оно спасает от главной патологии параболической интерполяции — серии всё уменьшающихся шагов, при которой метод формально движется, а отрезок неопределённости практически не сокращается. Требование «каждый второй шаг сокращает смещение хотя бы вдвое» делает такое зависание невозможным.



В результате метод Брента наследует надёжность золотого сечения (отрезок гарантированно сокращается геометрически) и скорость параболической интерполяции на гладких функциях вблизи минимума. Именно он реализован в `scipy.optimize.minimize_scalar` и вызывается там по умолчанию.

ЗАДАЧА 2

Метод Брента

1. Объясни, зачем нужны оба условия безопасности. Приведи пример ситуации, в которой первое условие выполнено, а второе нарушено.
2. Покажи, что при любом поведении функции длина отрезка неопределённости сокращается не медленнее, чем в золотом сечении, с точностью до постоянного множителя. Какая это скорость сходимости в худшем случае?
3. Для гладкой функции с $\varphi''(\alpha^*) > 0$ какую скорость сходимости следует ожидать вблизи минимума и почему?
4. (Практическая.) Реализуй дихотомию, золотое сечение и метод Брента с единым интерфейсом, считая число вызовов функции. Протестируй на

$$\varphi_1(\alpha) = (\alpha - 2)^2, \quad \varphi_2(\alpha) = |\alpha - 2|, \quad \varphi_3(\alpha) = \alpha^4 - 3\alpha^2 + \alpha,$$

построй график $\log |\alpha_k - \alpha^*|$ от числа вызовов функции. На какой из функций преимущество Брента исчезает?

3. ЕСЛИ КРАТКО

- Стоимость одномерного метода измеряется в вызовах функции: золотое сечение сокращает отрезок в 1,6 раза за вызов против 1,4 у дихотомии за счёт переиспользования пробной точки.
- Параболическая интерполяция быстра, но ненадёжна; золотое сечение надёжно, но медленно.
- Метод Брента страхует параболу золотым сечением двумя условиями безопасности и потому используется в библиотеках по умолчанию.